

```

require("dotenv").config();
const express = require("express");
const path = require("path");
const fs = require("fs");
const { Pool } = require("pg");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const QRCode = require("qrcode");
const { AsyncLocalStorage } = require("async_hooks");
const tenantContext = new AsyncLocalStorage();

const app = express();
app.set("trust proxy", 1);
app.use(express.json({limit:"2mb"}));
app.use(express.urlencoded({extended:true}));
// Frontend is served explicitly from the absolute filesystem path.
// This avoids any dependency on Express static-directory index resolution.
const PUBLIC_DIR = path.resolve(__dirname, "public");
const INDEX_HTML = path.join(PUBLIC_DIR, "index.html");
const INDEX_CONTENT = fs.readFileSync(INDEX_HTML, "utf8");

app.get(/^\/$/, (req, res) => {
  res.status(200).type("html").send(INDEX_CONTENT);
});
app.get(/^\/index\.html\/?$/, (req, res) => {
  res.status(200).type("html").send(INDEX_CONTENT);
});

// Static assets (logo, icons, manifest, service worker, etc.).
app.use(express.static(PUBLIC_DIR, { index: false }));
app.use("/api", (req,res,next)=>{ res.set("Cache-Control","no-store"); next(); })
app.use((req,res,next)=>{ res.set("X-Content-Type-Options","nosniff"); res.set("R

const pool = new Pool({connectionString: process.env.DATABASE_URL});

// Every authenticated request carries its tenant in AsyncLocalStorage.
// The query wrapper sets PostgreSQL session context on the same connection,
// allowing Row-Level Security to enforce tenant isolation server-side.
const rawPoolQuery = pool.query.bind(pool);
pool.query = async function tenantAwareQuery(text, params) {
  const ctx = tenantContext.getStore();
  if (!ctx?.shopId) return rawPoolQuery(text, params);
  const client = await pool.connect();
  try {

```

```

    await client.query("BEGIN");
    await client.query("SELECT set_config('app.shop_id',$1,true)", [String(ctx.sho
    const result = await client.query(text, params);
    await client.query("COMMIT");
    return result;
  } catch (e) {
    try { await client.query("ROLLBACK"); } catch {}
    throw e;
  } finally { client.release(); }
};

async function setTenant(client, shopId){
  await client.query("SELECT set_config('app.shop_id',$1,true)", [String(shopId)])
}
const JWT_SECRET = process.env.JWT_SECRET || "dev-only-change-me";
const PORT = Number(process.env.PORT || 3000);
const DEFAULT_ADMIN_PASSWORD = process.env.DEFAULT_ADMIN_PASSWORD || "Admin@12345

const DEFAULT_PERMS = {
  add_repair:true, view_repairs:true, edit_repair:true, change_status:true,
  customers:true, edit_customers:true, inventory:true, accounts:false,
  profits:false, reports:false, delete_repairs:false, delete_customers:false,
  users:false, settings:false, activity_logs:false, backups:false
};
const STATUSES = ["لا يصلح", "تم اصلاحه", "قيد اصلاح"];

async function q(text, params=[]){ return (await pool.query(text,params)).rows; }
async function one(text, params=[]){ return (await pool.query(text,params)).rows[
async function init(){
  const schemaCandidates = [
    path.join(__dirname, "db", "schema.sql"),
    path.join(__dirname, "BN-SMART-schema.sql"),
    path.join(__dirname, "src", "db", "schema.sql"),
  ];
  const schemaPath = schemaCandidates.find((p) => fs.existsSync(p));
  if (!schemaPath) {
    throw new Error(`Database schema not found. Checked: ${schemaCandidates.join(
  }
  const schema=fs.readFileSync(schemaPath,"utf8");
  await pool.query(schema);

  // SaaS foundation: create the tenant table and migrate legacy single-shop data
  // into one default tenant before enabling RLS.
  await pool.query(`CREATE TABLE IF NOT EXISTS shops (
    id BIGSERIAL PRIMARY KEY, slug VARCHAR(80) UNIQUE NOT NULL, name VARCHAR(160)
    logo TEXT, phone VARCHAR(40), address TEXT, currency VARCHAR(12) NOT NULL DEF
    receipt_settings JSONB NOT NULL DEFAULT '{}':::jsonb, created_at TIMESTAMPTZ N

```

```

    active BOOLEAN NOT NULL DEFAULT TRUE
  ));
const defaultShop=(await pool.query("SELECT * FROM shops WHERE slug='bn-smart'
  (await pool.query("INSERT INTO shops(slug,name,phone,address,currency) VALUES
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS due_at TIM
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS customer_r
await pool.query(`ALTER TABLE repair_orders ADD COLUMN IF NOT EXISTS received_a
await pool.query(`ALTER TABLE repair_parts ADD COLUMN IF NOT EXISTS sale_price

// Compatibility migration: older deployments may have been created from a sche
// that did not contain supplier_purchases. Create the table before any ALTER/I
// statements so startup is self-healing instead of failing with 42P01.
await pool.query(`CREATE TABLE IF NOT EXISTS supplier_purchases (
  id BIGSERIAL PRIMARY KEY,
  shop_id BIGINT,
  supplier VARCHAR(120) NOT NULL,
  items JSONB NOT NULL DEFAULT '[]'::jsonb,
  total_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  paid_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  notes TEXT,
  created_by BIGINT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
));

const tenantTables=['users','customers','repair_orders','repair_status_history'
for(const table of tenantTables){
  await pool.query(`ALTER TABLE ${table} ADD COLUMN IF NOT EXISTS shop_id BEGIN
  await pool.query(`UPDATE ${table} SET shop_id=$1 WHERE shop_id IS NULL`,`[defa
  await pool.query(`ALTER TABLE ${table} ALTER COLUMN shop_id SET NOT NULL`);
  await pool.query(`CREATE INDEX IF NOT EXISTS idx_${table}_shop_id ON ${table}
}
await pool.query(`ALTER TABLE supplier_purchases ADD COLUMN IF NOT EXISTS shop_
await pool.query(`UPDATE supplier_purchases SET shop_id=$1 WHERE shop_id IS NUL
await pool.query(`ALTER TABLE supplier_purchases ALTER COLUMN shop_id SET NOT N
await pool.query(`CREATE INDEX IF NOT EXISTS idx_supplier_purchases_shop_id ON
await pool.query(`ALTER TABLE users DROP CONSTRAINT IF EXISTS users_username_ke
await pool.query(`CREATE UNIQUE INDEX IF NOT EXISTS uq_users_shop_username ON u
const fkTables=['users',...tenantTables.filter(t=>t!=='users'),'supplier_purcha
for(const table of fkTables){
  const cname=`fk_${table}_shop`;
  await pool.query(`DO $$ BEGIN IF NOT EXISTS (SELECT 1 FROM pg_constraint WHER
}

// Settings used to be global. Migrate it once to tenant-scoped key/value stora
const settingsHasShop=await pool.query(`SELECT 1 FROM information_schema.column
if(!settingsHasShop.rows.length){
  await pool.query(`CREATE TABLE IF NOT EXISTS settings_tenant (id BIGSERIAL PR

```

```

    const oldSettings=await pool.query("SELECT key,value FROM settings");
    for(const row of oldSettings.rows){ await pool.query("INSERT INTO settings_te
    await pool.query("DROP TABLE settings");
    await pool.query("ALTER TABLE settings_tenant RENAME TO settings");
    await pool.query("CREATE INDEX IF NOT EXISTS idx_settings_shop_id ON settings
    }

const exists=(await pool.query("SELECT id FROM users WHERE shop_id=$1 LIMIT 1",
if(!exists){
    const hash=await bcrypt.hash(DEFAULT_ADMIN_PASSWORD,12);
    await pool.query("INSERT INTO users(shop_id,username,full_name,password_hash,
        [defaultShop.id,"admin","المدير الرئيسي",hash,JSON.stringify(Object.fromEnt
    }
const shopRow=await pool.query("SELECT value FROM settings WHERE shop_id=$1 AND
const shopValue=shopRow.rows[0]?.value && typeof shopRow.rows[0].value==='objec
const fixedShop={...shopValue,name:"BN SMART"};
await pool.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',$2)

// Enforce tenant isolation in PostgreSQL for every tenant-owned table.
const rlsTables=['users','customers','repair_orders','repair_status_history','p
for(const table of rlsTables){
    await pool.query(`ALTER TABLE ${table} ENABLE ROW LEVEL SECURITY`);
    await pool.query(`DROP POLICY IF EXISTS ${table}_tenant_isolation ON ${table}
    await pool.query(`CREATE POLICY ${table}_tenant_isolation ON ${table} USING (
    }

// Public tracking uses a narrowly-scoped SECURITY DEFINER function. It accepts
// only a tracking code and returns one repair, while authenticated APIs remain
await pool.query(`CREATE OR REPLACE FUNCTION public_track_repair(p_code TEXT) R
    id BIGINT, receipt_no INTEGER, tracking_code VARCHAR, brand VARCHAR, model VA
    updated_at TIMESTAMPTZ, due_at TIMESTAMPTZ, expected_price NUMERIC, paid_amou
) LANGUAGE SQL SECURITY DEFINER SET search_path=public AS $$
    SELECT r.id,r.receipt_no,r.tracking_code,r.brand,r.model,r.color,r.fault,r.st
    FROM repair_orders r JOIN customers c ON c.id=r.customer_id WHERE r.tracking_
    $$`);
await pool.query(`REVOKE ALL ON FUNCTION public_track_repair(TEXT) FROM PUBLIC`
await pool.query(`GRANT EXECUTE ON FUNCTION public_track_repair(TEXT) TO PUBLIC
await pool.query(`CREATE OR REPLACE FUNCTION public_track_history(p_repair_id B
    SELECT COALESCE(json_agg(x ORDER BY x.created_at),'[]'::json) FROM (SELECT st
    $$`);
await pool.query(`REVOKE ALL ON FUNCTION public_track_history(BIGINT) FROM PUBL
await pool.query(`GRANT EXECUTE ON FUNCTION public_track_history(BIGINT) TO PUB
}

function auth(req,res,next){
    const h=req.headers.authorization||"";
    const token=h.startsWith("Bearer ")?h.slice(7):null;
    if(!token)return res.status(401).json({error:"تسجيل الدخول مطلوب"});

```

```

    try{
        req.user=jwt.verify(token,JWT_SECRET);
        if(!req.user.shop_id)return res.status(401).json({error:"لا تحتوي على محل صالح |
        tenantContext.run({shopId:req.user.shop_id},()=>next());
    }catch(e){res.status(401).json({error:"الجلسة غير صالحة"})}
}

function manager(req,res,next){ if(req.user.role!=="manager")return res.status(40
function can(permission){return (req,res,next)=>{ if(req.user.role==="manager" ||
async function log(userId,action,repairId=null,details={}){const ctx=tenantContex

app.post("/api/auth/register-shop",async(req,res)=>{
    const b=req.body||{};
    const shopName=String(b.shopName||"").trim();
    const slug=String(b.slug||shopName).trim().toLowerCase().replace(/[a-z0-9-]+/g
    const username=String(b.username||"admin").trim();
    const password=String(b.password||"");
    if(shopName.length<2 || !slug || username.length<2 || password.length<8) return
    const client=await pool.connect();
    try{
        await client.query("BEGIN");
        const shop=(await client.query("INSERT INTO shops(slug,name,phone,address,cur
        await setTenant(client,shop.id);
        const hash=await bcrypt.hash(password,12);
        const permissions=Object.fromEntries(Object.keys(DEFAULT_PERMS).map(k=>[k,tru
        const user=(await client.query("INSERT INTO users(shop_id,username,full_name,
        await client.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',
        await client.query("COMMIT");
        res.status(201).json({shop:{id:shop.id,slug:shop.slug,name:shop.name},user});
    }catch(e){try{await client.query("ROLLBACK")}catch{}}; if(e?.code==='23505') ret
});

app.post("/api/auth/login",async(req,res)=>{
    try{
        const {username,password,shopSlug}=req.body;
        const slug=String(shopSlug||"bn-smart").trim().toLowerCase();
        const shop=await one("SELECT id,slug,name,active FROM shops WHERE slug=$1",[s
        if(!shop || !shop.active) return res.status(401).json({error:"جود أو غير مفع
        const u=(await pool.query("SELECT * FROM users WHERE shop_id=$1 AND username=
        if(!u || !u.active || !(await bcrypt.compare(password||"",u.password_hash)))
        await tenantContext.run({shopId:shop.id},()=>pool.query("UPDATE users SET las
        const permissions=u.role==="manager"?Object.fromEntries(Object.keys(DEFAULT_P
        const token=jwt.sign({id:u.id,username:u.username,full_name:u.full_name,role:
        res.json({token,user:{id:u.id,username:u.username,full_name:u.full_name,role:
    }catch(e){res.status(500).json({error:"خطأ في الخادم"})}
});

app.get("/api/auth/me",auth,(req,res)=>res.json({user:req.user}));
app.get("/health",(req,res)=>res.json({ok:true,service:"BN SMART"}));

```

```

app.get("/api/dashboard",auth,can("view_repairs"),async(req,res)=>{
  const stats=await one(`SELECT
    COUNT(*) FILTER(WHERE status='فيد الاصلاح') AS in_repair,
    COUNT(*) FILTER(WHERE status='تم اصلاحه') AS repaired,
    COUNT(*) FILTER(WHERE status='لايصلح') AS not_repairable,
    COUNT(*) FILTER(WHERE created_at::date=CURRENT_DATE) AS received_today,
    COUNT(*) FILTER(WHERE customer_received=TRUE) AS customer_received_count,
    COALESCE(SUM(paid_amount),0) AS revenue,
    COALESCE(SUM(expected_price-part_cost) FILTER(WHERE customer_received=TRUE),0
    COALESCE(SUM(expected_price-paid_amount),0) AS remaining
  FROM repair_orders`);
  const recent=await q(`SELECT r.*,c.name customer_name,c.phone FROM repair_order
  const statuses=await q("SELECT status,COUNT(*) count FROM repair_orders GROUP B
  res.json({stats,recent,statuses});
});

app.get("/api/reports/daily",auth,can("reports"),async(req,res)=>{
  try{
    const period=String(req.query.period||"30");
    let where="";
    const params=[];
    if(period!="all"){
      const days=Math.max(1,Math.min(3650,Number(period)||30));
      params.push(days);
      where=`WHERE r.created_at >= CURRENT_DATE - ($1::int - 1) * INTERVAL '1 day
    }
    const rows=await q(`SELECT r.created_at::date AS day, COUNT(*)::int AS repair
      COALESCE(SUM(r.paid_amount),0)::numeric AS paid,
      COALESCE(SUM(r.expected_price-r.part_cost) FILTER (WHERE r.customer_receive
      FROM repair_orders r ${where}
      GROUP BY r.created_at::date ORDER BY repairs_count DESC, day DESC`,params);
    const total=rows.reduce((n,r)=>n+Number(r.repairs_count||0),0);
    const totalPaid=rows.reduce((n,r)=>n+Number(r.paid||0),0);
    const totalProfit=rows.reduce((n,r)=>n+Number(r.profit||0),0);
    const daysSpan=period=="all"?Math.max(rows.length,1):Math.max(1,Number(perio
    res.json({
      total_repairs: total,
      total_paid: totalPaid,
      total_profit: totalProfit,
      average_daily_repairs: total/daysSpan,
      days: rows.map(r=>({...r,percent: total?Number(r.repairs_count)*100/total:0
    });
  }catch(e){console.error(e);res.status(500).json({error:"تعذر تحميل تقرير الأيام"
});

app.get("/api/repairs",auth,can("view_repairs"),async(req,res)=>{

```

```

const {search="",status=""}=req.query;
const params=[];let where=[];
if(status){params.push(status);where.push(`r.status=${params.length}`)}
if(search){params.push(`%${search}%`);where.push(`(r.receipt_no::text ILIKE ${search})`)}
const sql=`SELECT r.*,c.name customer_name,c.phone customer_phone,u.full_name c
FROM repair_orders r JOIN customers c ON c.id=r.customer_id LEFT JOIN users u ON u.id=r.created_by
${where.length?"WHERE "+where.join(" AND ")":""} ORDER BY r.created_at`;
res.json(await q(sql,params));
});
app.get("/api/repairs/:id",auth,can("view_repairs"),async(req,res)=>{
const r=await one(`SELECT r.*,c.name customer_name,c.phone customer_phone,c.id
cu.full_name created_by_name,uu.full_name updated_by_name
FROM repair_orders r JOIN customers c ON c.id=r.customer_id
LEFT JOIN users cu ON cu.id=r.created_by LEFT JOIN users uu ON uu.id=r.updated_by`);
if(!r)return res.status(404).json({error:"الإصلاح غير موجود"});
r.history=await q(`SELECT h.*,u.full_name changed_by_name FROM repair_status_history h JOIN users u ON u.id=h.changed_by`);
r.payments=await q(`SELECT p.*,u.full_name recorded_by_name FROM payments p LEFT JOIN users u ON u.id=p.recorded_by`);
r.parts=await q(`SELECT rp.*,sp.name part_name FROM repair_parts rp JOIN spare_parts sp ON sp.id=rp.part_id`);
res.json(r);
});
app.post("/api/repairs",auth,can("add_repair"),async(req,res)=>{
const client=await pool.connect();
try{
await client.query("BEGIN");
await setTenant(client,req.user.shop_id);
const b=req.body;
const customerName=String(b.customerName||"عميل").trim()||"عميل";
const phone=String(b.phone||"").trim();
let customerId;
// Never rename an existing customer when creating a new repair.
// Empty phone numbers never match another customer.
if(phone){
const c=await client.query("SELECT id FROM customers WHERE phone=$1 ORDER BY id");
if(c.rows[0]) customerId=c.rows[0].id;
else customerId=(await client.query("INSERT INTO customers(shop_id,name,phone) VALUES($1,$2,$3) RETURNING id"))[0].id;
}else{
customerId=(await client.query("INSERT INTO customers(shop_id,name,phone) VALUES($1,$2,$3) RETURNING id"))[0].id;
}
await client.query("SELECT pg_advisory_xact_lock(1842026)");
const last=await client.query("SELECT COALESCE(MAX(receipt_no),184)+1 n FROM repair_orders");
const no=last.rows[0].n, tracking="QF-"+String(no).padStart(4,"0");
const r=(await client.query(`INSERT INTO repair_orders(shop_id,receipt_no,tracking,brand,model,color,power,created_by,updated_by)
VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16,$17,$18,$19,$20,$21,$22,$23,$24,$25,$26,$27,$28,$29,$30,$31,$32,$33,$34,$35,$36,$37,$38,$39,$40,$41,$42,$43,$44,$45,$46,$47,$48,$49,$50,$51,$52,$53,$54,$55,$56,$57,$58,$59,$60,$61,$62,$63,$64,$65,$66,$67,$68,$69,$70,$71,$72,$73,$74,$75,$76,$77,$78,$79,$80,$81,$82,$83,$84,$85,$86,$87,$88,$89,$90,$91,$92,$93,$94,$95,$96,$97,$98,$99,$100,$101,$102,$103,$104,$105,$106,$107,$108,$109,$110,$111,$112,$113,$114,$115,$116,$117,$118,$119,$120,$121,$122,$123,$124,$125,$126,$127,$128,$129,$130,$131,$132,$133,$134,$135,$136,$137,$138,$139,$140,$141,$142,$143,$144,$145,$146,$147,$148,$149,$150,$151,$152,$153,$154,$155,$156,$157,$158,$159,$160,$161,$162,$163,$164,$165,$166,$167,$168,$169,$170,$171,$172,$173,$174,$175,$176,$177,$178,$179,$180,$181,$182,$183,$184,$185,$186,$187,$188,$189,$190,$191,$192,$193,$194,$195,$196,$197,$198,$199,$200,$201,$202,$203,$204,$205,$206,$207,$208,$209,$210,$211,$212,$213,$214,$215,$216,$217,$218,$219,$220,$221,$222,$223,$224,$225,$226,$227,$228,$229,$230,$231,$232,$233,$234,$235,$236,$237,$238,$239,$240,$241,$242,$243,$244,$245,$246,$247,$248,$249,$250,$251,$252,$253,$254,$255,$256,$257,$258,$259,$260,$261,$262,$263,$264,$265,$266,$267,$268,$269,$270,$271,$272,$273,$274,$275,$276,$277,$278,$279,$280,$281,$282,$283,$284,$285,$286,$287,$288,$289,$290,$291,$292,$293,$294,$295,$296,$297,$298,$299,$300,$301,$302,$303,$304,$305,$306,$307,$308,$309,$310,$311,$312,$313,$314,$315,$316,$317,$318,$319,$320,$321,$322,$323,$324,$325,$326,$327,$328,$329,$330,$331,$332,$333,$334,$335,$336,$337,$338,$339,$340,$341,$342,$343,$344,$345,$346,$347,$348,$349,$350,$351,$352,$353,$354,$355,$356,$357,$358,$359,$360,$361,$362,$363,$364,$365,$366,$367,$368,$369,$370,$371,$372,$373,$374,$375,$376,$377,$378,$379,$380,$381,$382,$383,$384,$385,$386,$387,$388,$389,$390,$391,$392,$393,$394,$395,$396,$397,$398,$399,$400,$401,$402,$403,$404,$405,$406,$407,$408,$409,$410,$411,$412,$413,$414,$415,$416,$417,$418,$419,$420,$421,$422,$423,$424,$425,$426,$427,$428,$429,$430,$431,$432,$433,$434,$435,$436,$437,$438,$439,$440,$441,$442,$443,$444,$445,$446,$447,$448,$449,$450,$451,$452,$453,$454,$455,$456,$457,$458,$459,$460,$461,$462,$463,$464,$465,$466,$467,$468,$469,$470,$471,$472,$473,$474,$475,$476,$477,$478,$479,$480,$481,$482,$483,$484,$485,$486,$487,$488,$489,$490,$491,$492,$493,$494,$495,$496,$497,$498,$499,$500,$501,$502,$503,$504,$505,$506,$507,$508,$509,$510,$511,$512,$513,$514,$515,$516,$517,$518,$519,$520,$521,$522,$523,$524,$525,$526,$527,$528,$529,$530,$531,$532,$533,$534,$535,$536,$537,$538,$539,$540,$541,$542,$543,$544,$545,$546,$547,$548,$549,$550,$551,$552,$553,$554,$555,$556,$557,$558,$559,$560,$561,$562,$563,$564,$565,$566,$567,$568,$569,$570,$571,$572,$573,$574,$575,$576,$577,$578,$579,$580,$581,$582,$583,$584,$585,$586,$587,$588,$589,$590,$591,$592,$593,$594,$595,$596,$597,$598,$599,$600,$601,$602,$603,$604,$605,$606,$607,$608,$609,$610,$611,$612,$613,$614,$615,$616,$617,$618,$619,$620,$621,$622,$623,$624,$625,$626,$627,$628,$629,$630,$631,$632,$633,$634,$635,$636,$637,$638,$639,$640,$641,$642,$643,$644,$645,$646,$647,$648,$649,$650,$651,$652,$653,$654,$655,$656,$657,$658,$659,$660,$661,$662,$663,$664,$665,$666,$667,$668,$669,$670,$671,$672,$673,$674,$675,$676,$677,$678,$679,$680,$681,$682,$683,$684,$685,$686,$687,$688,$689,$690,$691,$692,$693,$694,$695,$696,$697,$698,$699,$700,$701,$702,$703,$704,$705,$706,$707,$708,$709,$710,$711,$712,$713,$714,$715,$716,$717,$718,$719,$720,$721,$722,$723,$724,$725,$726,$727,$728,$729,$730,$731,$732,$733,$734,$735,$736,$737,$738,$739,$740,$741,$742,$743,$744,$745,$746,$747,$748,$749,$750,$751,$752,$753,$754,$755,$756,$757,$758,$759,$760,$761,$762,$763,$764,$765,$766,$767,$768,$769,$770,$771,$772,$773,$774,$775,$776,$777,$778,$779,$780,$781,$782,$783,$784,$785,$786,$787,$788,$789,$790,$791,$792,$793,$794,$795,$796,$797,$798,$799,$800,$801,$802,$803,$804,$805,$806,$807,$808,$809,$810,$811,$812,$813,$814,$815,$816,$817,$818,$819,$820,$821,$822,$823,$824,$825,$826,$827,$828,$829,$830,$831,$832,$833,$834,$835,$836,$837,$838,$839,$840,$841,$842,$843,$844,$845,$846,$847,$848,$849,$850,$851,$852,$853,$854,$855,$856,$857,$858,$859,$860,$861,$862,$863,$864,$865,$866,$867,$868,$869,$870,$871,$872,$873,$874,$875,$876,$877,$878,$879,$880,$881,$882,$883,$884,$885,$886,$887,$888,$889,$890,$891,$892,$893,$894,$895,$896,$897,$898,$899,$900,$901,$902,$903,$904,$905,$906,$907,$908,$909,$910,$911,$912,$913,$914,$915,$916,$917,$918,$919,$920,$921,$922,$923,$924,$925,$926,$927,$928,$929,$930,$931,$932,$933,$934,$935,$936,$937,$938,$939,$940,$941,$942,$943,$944,$945,$946,$947,$948,$949,$950,$951,$952,$953,$954,$955,$956,$957,$958,$959,$960,$961,$962,$963,$964,$965,$966,$967,$968,$969,$970,$971,$972,$973,$974,$975,$976,$977,$978,$979,$980,$981,$982,$983,$984,$985,$986,$987,$988,$989,$990,$991,$992,$993,$994,$995,$996,$997,$998,$999,$1000,$1001,$1002,$1003,$1004,$1005,$1006,$1007,$1008,$1009,$1010,$1011,$1012,$1013,$1014,$1015,$1016,$1017,$1018,$1019,$1020,$1021,$1022,$1023,$1024,$1025,$1026,$1027,$1028,$1029,$1030,$1031,$1032,$1033,$1034,$1035,$1036,$1037,$1038,$1039,$1040,$1041,$1042,$1043,$1044,$1045,$1046,$1047,$1048,$1049,$1050,$1051,$1052,$1053,$1054,$1055,$1056,$1057,$1058,$1059,$1060,$1061,$1062,$1063,$1064,$1065,$1066,$1067,$1068,$1069,$1070,$1071,$1072,$1073,$1074,$1075,$1076,$1077,$1078,$1079,$1080,$1081,$1082,$1083,$1084,$1085,$1086,$1087,$1088,$1089,$1090,$1091,$1092,$1093,$1094,$1095,$1096,$1097,$1098,$1099,$1100,$1101,$1102,$1103,$1104,$1105,$1106,$1107,$1108,$1109,$1110,$1111,$1112,$1113,$1114,$1115,$1116,$1117,$1118,$1119,$1120,$1121,$1122,$1123,$1124,$1125,$1126,$1127,$1128,$1129,$1130,$1131,$1132,$1133,$1134,$1135,$1136,$1137,$1138,$1139,$1140,$1141,$1142,$1143,$1144,$1145,$1146,$1147,$1148,$1149,$1150,$1151,$1152,$1153,$1154,$1155,$1156,$1157,$1158,$1159,$1160,$1161,$1162,$1163,$1164,$1165,$1166,$1167,$1168,$1169,$1170,$1171,$1172,$1173,$1174,$1175,$1176,$1177,$1178,$1179,$1180,$1181,$1182,$1183,$1184,$1185,$1186,$1187,$1188,$1189,$1190,$1191,$1192,$1193,$1194,$1195,$1196,$1197,$1198,$1199,$1200,$1201,$1202,$1203,$1204,$1205,$1206,$1207,$1208,$1209,$1210,$1211,$1212,$1213,$1214,$1215,$1216,$1217,$1218,$1219,$1220,$1221,$1222,$1223,$1224,$1225,$1226,$1227,$1228,$1229,$1230,$1231,$1232,$1233,$1234,$1235,$1236,$1237,$1238,$1239,$1240,$1241,$1242,$1243,$1244,$1245,$1246,$1247,$1248,$1249,$1250,$1251,$1252,$1253,$1254,$1255,$1256,$1257,$1258,$1259,$1260,$1261,$1262,$1263,$1264,$1265,$1266,$1267,$1268,$1269,$1270,$1271,$1272,$1273,$1274,$1275,$1276,$1277,$1278,$1279,$1280,$1281,$1282,$1283,$1284,$1285,$1286,$1287,$1288,$1289,$1290,$1291,$1292,$1293,$1294,$1295,$1296,$1297,$1298,$1299,$1300,$1301,$1302,$1303,$1304,$1305,$1306,$1307,$1308,$1309,$1310,$1311,$1312,$1313,$1314,$1315,$1316,$1317,$1318,$1319,$1320,$1321,$1322,$1323,$1324,$1325,$1326,$1327,$1328,$1329,$1330,$1331,$1332,$1333,$1334,$1335,$1336,$1337,$1338,$1339,$1340,$1341,$1342,$1343,$1344,$1345,$1346,$1347,$1348,$1349,$1350,$1351,$1352,$1353,$1354,$1355,$1356,$1357,$1358,$1359,$1360,$1361,$1362,$1363,$1364,$1365,$1366,$1367,$1368,$1369,$1370,$1371,$1372,$1373,$1374,$1375,$1376,$1377,$1378,$1379,$1380,$1381,$1382,$1383,$1384,$1385,$1386,$1387,$1388,$1389,$1390,$1391,$1392,$1393,$1394,$1395,$1396,$1397,$1398,$1399,$1400,$1401,$1402,$1403,$1404,$1405,$1406,$1407,$1408,$1409,$1410,$1411,$1412,$1413,$1414,$1415,$1416,$1417,$1418,$1419,$1420,$1421,$1422,$1423,$1424,$1425,$1426,$1427,$1428,$1429,$1430,$1431,$1432,$1433,$1434,$1435,$1436,$1437,$1438,$1439,$1440,$1441,$1442,$1443,$1444,$1445,$1446,$1447,$1448,$1449,$1450,$1451,$1452,$1453,$1454,$1455,$1456,$1457,$1458,$1459,$1460,$1461,$1462,$1463,$1464,$1465,$1466,$1467,$1468,$1469,$1470,$1471,$1472,$1473,$1474,$1475,$1476,$1477,$1478,$1479,$1480,$1481,$1482,$1483,$1484,$1485,$1486,$1487,$1488,$1489,$1490,$1491,$1492,$1493,$1494,$1495,$1496,$1497,$1498,$1499,$1500,$1501,$1502,$1503,$1504,$1505,$1506,$1507,$1508,$1509,$1510,$1511,$1512,$1513,$1514,$1515,$1516,$1517,$1518,$1519,$1520,$1521,$1522,$1523,$1524,$1525,$1526,$1527,$1528,$1529,$1530,$1531,$1532,$1533,$1534,$1535,$1536,$1537,$1538,$1539,$1540,$1541,$1542,$1543,$1544,$1545,$1546,$1547,$1548,$1549,$1550,$1551,$1552,$1553,$1554,$1555,$1556,$1557,$1558,$1559,$1560,$1561,$1562,$1563,$1564,$1565,$1566,$1567,$1568,$1569,$1570,$1571,$1572,$1573,$1574,$1575,$1576,$1577,$1578,$1579,$1580,$1581,$1582,$1583,$1584,$1585,$1586,$1587,$1588,$1589,$1590,$1591,$1592,$1593,$1594,$1595,$1596,$1597,$1598,$1599,$1600,$1601,$1602,$1603,$1604,$1605,$1606,$1607,$1608,$1609,$1610,$1611,$1612,$1613,$1614,$1615,$1616,$1617,$1618,$1619,$1620,$1621,$1622,$1623,$1624,$1625,$1626,$1627,$1628,$1629,$1630,$1631,$1632,$1633,$1634,$1635,$1636,$1637,$1638,$1639,$1640,$1641,$1642,$1643,$1644,$1645,$1646,$1647,$1648,$1649,$1650,$1651,$1652,$1653,$1654,$1655,$1656,$1657,$1658,$1659,$1660,$1661,$1662,$1663,$1664,$1665,$1666,$1667,$1668,$1669,$1670,$1671,$1672,$1673,$1674,$1675,$1676,$1677,$1678,$1679,$1680,$1681,$1682,$1683,$1684,$1685,$1686,$1687,$1688,$1689,$1690,$1691,$1692,$1693,$1694,$1695,$1696,$1697,$1698,$1699,$1700,$1701,$1702,$1703,$1704,$1705,$1706,$1707,$1708,$1709,$1710,$1711,$1712,$1713,$1714,$1715,$1716,$1717,$1718,$1719,$1720,$1721,$1722,$1723,$1724,$1725,$1726,$1727,$1728,$1729,$1730,$1731,$1732,$1733,$1734,$1735,$1736,$1737,$1738,$1739,$1740,$1741,$1742,$1743,$1744,$1745,$1746,$1747,$1748,$1749,$1750,$1751,$1752,$1753,$1754,$1755,$1756,$1757,$1758,$1759,$1760,$1761,$1762,$1763,$1764,$1765,$1766,$1767,$1768,$1769,$1770,$1771,$1772,$1773,$1774,$1775,$1776,$1777,$1778,$1779,$1780,$1781,$1782,$1783,$1784,$1785,$1786,$1787,$1788,$1789,$1790,$1791,$1792,$1793,$1794,$1795,$1796,$1797,$1798,$1799,$1800,$1801,$1802,$1803,$1804,$1805,$1806,$1807,$1808,$1809,$1810,$1811,$1812,$1813,$1814,$1815,$1816,$1817,$1818,$1819,$1820,$1821,$1822,$1823,$1824,$1825,$1826,$1827,$1828,$1829,$1830,$1831,$1832,$1833,$1834,$1835,$1836,$1837,$1838,$1839,$1840,$1841,$1842,$1843,$1844,$1845,$1846,$1847,$1848,$1849,$1850,$1851,$1852,$1853,$1854,$1855,$1856,$1857,$1858,$1859,$1860,$1861,$1862,$1863,$1864,$1865,$1866,$1867,$1868,$1869,$1870,$1871,$1872,$1873,$1874,$1875,$1876,$1877,$1878,$1879,$1880,$1881,$1882,$1883,$1884,$1885,$1886,$1887,$1888,$1889,$1890,$1891,$1892,$1893,$1894,$1895,$1896,$1897,$1898,$1899,$1900,$1901,$1902,$1903,$1904,$1905,$1906,$1907,$1908,$1909,$1910,$1911,$1912,$1913,$1914,$1915,$1916,$1917,$1918,$1919,$1920,$1921,$1922,$1923,$1924,$1925,$1926,$1927,$1928,$1929,$1930,$1931,$1932,$1933,$1934,$1935,$1936,$1937,$1938,$1939,$1940,$1941,$1942,$1943,$1944,$1945,$1946,$1947,$1948,$1949,$1950,$1951,$1952,$1953,$1954,$1955,$1956,$1957,$1958,$1959,$1960,$1961,$1962,$1963,$1964,$1965,$1966,$1967,$1968,$1969,$1970,$1971,$1972,$1973,$1974,$1975,$1976,$1977,$1978,$1979,$1980,$1981,$1982,$1983,$1984,$1985,$1986,$1987,$1988,$1989,$1990,$1991,$1992,$1993,$1994,$1995,$1996,$1997,$1998,$1999,$2000,$2001,$2002,$2003,$2004,$2005,$2006,$2007,$2008,$2009,$2010,$2011,$2012,$2013,$2014,$2015,$2016,$2017,$2018,$2019,$2020,$2021,$2022,$2023,$2024,$2025,$2026,$2027,$2028,$2029,$2030,$2031,$2032,$2033,$2034,$2035,$2036,$2037,$2038,$2039,$2040,$2041,$2042,$2043,$2044,$2045,$2046,$2047,$2048,$2049,$2050,$2051,$2052,$2053,$2054,$2055,$2056,$2057,$2058,$2059,$2060,$2061,$2062,$2063,$2064,$2065,$2066,$2067,$2068,$2069,$2070,$2071,$2072,$2073,$2074,$2075,$2076,$2077,$2078,$2079,$2080,$2081,$2082,$2083,$2084,$2085,$2086,$2087,$2088,$2089,$2090,$2091,$2092,$2093,$2094,$2095,$2096,$2097,$2098,$2099,$2100,$2101,$2102,$2103,$2104,$2105,$2106,$2107,$2108,$2109,$2110,$2111,$2112,$2113,$2114,$2115,$2116,$2117,$2118,$2119,$2120,$2121,$2122,$2123,$2124,$2125,$2126,$2127,$2128,$2129,$2130,$2131,$2132,$2133,$2134,$2135,$2136,$2137,$2138,$2139,$2140,$2141,$2142,$2143,$2144,$2145,$2146,$2147,$2148,$2149,$2150,$2151,$2152,$2153,$2154,$2155,$2156,$2157,$2158,$2159,$2160,$2161,$2162,$2163,$2164,$2165,$2166,$2167,$2168,$2169,$2170,$2171,$2172,$2173,$2174,$2175,$2176,$2177,$2178,$2179,$2180,$2181,$2182,$2183,$2184,$2185,$2186,$2187,$2188,$2189,$2190,$2191,$2192,$2193,$2194,$2195,$2196,$2197,$2198,$2199,$2200,$2201,$2202,$2203,$2204,$2205,$2206,$2207,$2208,$2209,$2210,$2211,$2212,$2213,$2214,$2215,$2216,$2217,$2218,$2219,$2220,$2221,$2222,$2223,$2224,$2225,$2226,$2227,$2228,$2229,$2230,$2231,$2232,$2233,$2234,$2235,$2236,$2237,$2238,$2239,$2240,$2241,$2242,$2243,$2244,$2245,$2246,$2247,$2248,$2249,$2250,$2251,$2252,$2253,$2254,$2255,$2256,$2257,$2258,$2259,$2260,$2261,$2262,$2263,$2264,$2265,$2266,$2267,$2268,$2269,$2270,$2271,$2272,$2273,$2274,$2275,$2276,$2277,$2278,$2279,$2280,$2281,$2282,$2283,$2284,$2285,$2286,$2287,$2288,$2289,$2290,$2291,$2292,$2293,$2294,$2295,$2296,$2297,$2298,$2299,$2300,$2301,$2302,$2303,$2304,$2305,$2306,$2307,$2308,$2309,$2310,$2311,$2312,$2313,$2314,$2315,$2316,$2317,$2318,$2319,$2320,$2321,$2322,$2323,$2324,$2325,$2326,$2327,$2328,$2329,$2330,$2331,$2332,$2333,$2334,$2335,$2336,$2337,$2338,$2339,$2340,$2341,$2342,$2343,$2344,$2345,$2346,$2347,$2348,$2349,$2350,$2351,$2352,$2353,$2354,$2355,$2356,$2357,$2358,$2359,$2360,$2361,$2362,$2363,$2364,$2365,$2366,$2367,$2368,$2369,$2370,$2371,$2372,$2373,$2374,$2375,$2376,$2377,$2378,$2379,$2380,$2381,$2382,$2383,$2384,$2385,$2386,$2387,$2388,$2389,$2390,$2391,$2392,$2393,$2394,$2395,$2396,$2397,$2398,$2399,$2400,$2401,$2402,$2403,$2404,$2405,$2406,$2407,$2408,$2409,$2410,$2411,$2412,$2413,$2414,$2415,$2416,$2417,$2418,$2419,$2420,$2421,$2422,$2423,$2424,$2425,$2426,$2427,$2428,$2429,$2430,$2431,$2432,$2433,$2434,$2435,$2436,$2437,$2438,$2439,$2440,$2441,$2442,$2443,$2444,$2445,$2446,$2447,$2448,$2449,$2450,$2451,$2452,$2453,$2454,$2455,$2456,$2457,$2458,$2459,$2460,$2461,$2462,$2463,$2464,$2465,$2466,$2467,$2468,$2469,$2470,$2471,$2472,$2473,$2474,$2475,$2476,$2477,$2478,$2479,$2480,$2481,$2482,$2483,$2484,$2485,$2486,$2487,$2488,$2489,$2490,$2491,$2492,$2493,$2494,$2495,$2496,$2497,$2498,$2499,$2500,$2501,$2502,$2503,$2504,$2505,$2506,$2507,$2508,$2509,$2510,$2511,$2512,$2513,$2514,$2515,$2516,$2517,$2518,$2519,$2520,$2521,$2522,$2523,$2524,$2525,$2526,$2527,$2528,$2529,$2530,$2531,$2532,$2533,$2534,$2535,$2536,$2537,$2538,$2539,$2540,$2541,$2542,$2543,$2544,$2545,$2546,$2547,$2548,$2549,$2550,$2551
```

```

    await log(req.user.id, "إضافة وصل", r.id, {receipt_no: no});
    res.status(201).json(r);
  } catch (e) {
    await client.query("ROLLBACK");
    console.error(e);
    res.status(500).json({});
  }
});

app.patch("/api/repairs/:id", auth, can("edit_repair"), async (req, res) => {
  const b = req.body;
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    await setTenant(client, req.user.shop_id);
    const existing = await client.query("SELECT * FROM repair_orders WHERE id=$1");
    if (!existing.rows[0]) {
      await client.query("ROLLBACK");
      return res.status(404).json({});
    }
    const old = existing.rows[0];
    let customerId = old.customer_id;
    if (b.customerName !== undefined || b.phone !== undefined) {
      const oldCustomer = (await client.query("SELECT id, name, phone FROM customers WHERE id=$1")).rows[0];
      const newName = String(b.customerName === undefined ? (oldCustomer?.name || "عميل") : b.customerName);
      const newPhone = String(b.phone === undefined ? (oldCustomer?.phone || "") : b.phone);
      const oldName = String(oldCustomer?.name || "").trim();
      const oldPhone = String(oldCustomer?.phone || "").trim();
      if (newName !== oldName || newPhone !== oldPhone) {
        // Never mutate a shared customer record when editing one repair.
        const same = await client.query("SELECT id FROM customers WHERE name=$1 AND phone=$2");
        if (same.rows[0]) {
          customerId = same.rows[0].id;
        } else {
          await client.query("INSERT INTO customers (shop_id, name, phone) VALUES ($1, $2, $3)");
        }
      }
    }
    const r = (await client.query(`UPDATE repair_orders SET
      customer_id=$1, brand=$2, model=$3, color=$4, due_at=COALESCE($5, due_at),
      power_state=$7, fault=$8, diagnosis=$9, expected_price=COALESCE($10, expected_price),
      part_cost=COALESCE($12, part_cost), accessories=COALESCE($13, accessories),
      notes=COALESCE($15, notes), supplier=COALESCE($16, supplier), updated_by=$17,
      [customerId, b.brand, b.model, b.color, b.dueAt === undefined ? null : b.dueAt, b.powerState,
      b.price === undefined ? null : Number(b.price), b.paid === undefined ? null : Number(b.paid),
      b.accessories === null ? null : JSON.stringify(b.accessories), b.accessoryNotes, b.notes]`)).rows[0];
    await client.query("COMMIT");
    await log(req.user.id, "تعديل بيانات الوصل", r.id, {changed_fields: Object.keys(b).filter(k => b[k] !== r[k])});
  } catch (e) {
    await client.query("ROLLBACK");
    console.error(e);
    res.status(500).json({});
  }
});

app.get("/api/repairs/:id/parts", auth, can("view_repairs"), async (req, res) => {
  res.json(await q(`SELECT rp.*, sp.name part_name FROM repair_parts rp JOIN spare_parts sp ON rp.spare_part_id=sp.id WHERE rp.repair_order_id=$1`));
});

app.post("/api/repairs/:id/parts", auth, can("edit_repair"), async (req, res) => {
  const name = String(req.body?.name || "").trim();
  const qty = Math.max(1, Number(req.body?.qty));
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    await setTenant(client, req.user.shop_id);
    const r = (await client.query("SELECT id FROM repair_orders WHERE id=$1")).rows[0];
    if (!r) {
      await client.query("ROLLBACK");
      return res.status(404).json({});
    }
    const oldPart = (await client.query("SELECT * FROM repair_parts WHERE repair_order_id=$1 AND spare_part_id=$2")).rows[0];
    if (!oldPart) {
      await client.query("ROLLBACK");
      return res.status(404).json({});
    }
    const newPart = {
      repair_order_id: r.id,
      spare_part_id: req.body?.spare_part_id || oldPart.spare_part_id,
      qty: req.body?.qty || oldPart.qty,
      name: name,
      price: req.body?.price || oldPart.price,
      part_name: name
    };
    await client.query("INSERT INTO repair_parts (repair_order_id, spare_part_id, qty, name, price, part_name) VALUES ($1, $2, $3, $4, $5, $6)");
    await client.query("COMMIT");
    await log(req.user.id, "إضافة قطعة", r.id, {part_name: name, qty: req.body?.qty});
  } catch (e) {
    await client.query("ROLLBACK");
    console.error(e);
    res.status(500).json({});
  }
});

```

```

    if(!name) return res.status(400).json({error:'اسم القطعة مطلوب'});
    const client=await pool.connect(); try{await client.query('BEGIN');await setTen
    const sp=(await client.query(`INSERT INTO spare_parts(shop_id,name,quantity,c
    const rp=(await client.query(`INSERT INTO repair_parts(shop_id,repair_id,part
    await client.query(`UPDATE repair_orders SET part_cost=COALESCE(part_cost,0)+
    await client.query('COMMIT'); res.status(201).json({...rp,part_name:name});
    }catch(e){await client.query('ROLLBACK');console.error(e);res.status(500).json(
  ));
  app.delete("/api/repairs/:id/parts/:partId",auth,can("edit_repair"),async(req,res
  app.delete("/api/repairs/:id",auth,can("delete_repairs"),async(req,res)=>{
    try{
      const r=await one("SELECT id,receipt_no,tracking_code FROM repair_orders WHER
      if(!r) return res.status(404).json({error:"الوصل غير موجود"});
      await log(req.user.id,"حذف الوصل",r.id,{receipt_no:r.receipt_no,tracking_code
      await pool.query("DELETE FROM repair_orders WHERE id=$1",[r.id]);
      res.json({ok:true});
    }catch(e){console.error(e);res.status(500).json({error:"تعذر حذف الوصل"})}
  });

  app.patch("/api/repairs/:id/status",auth,can("change_status"),async(req,res)=>{
    const status=req.body.status;
    if(!STATUSES.includes(status)) return res.status(400).json({error:"الـة غير صالحة"});
    const r=await one("UPDATE repair_orders SET status=$1,updated_by=$2,updated_at=
    if(!r) return res.status(404).json({error:"غير موجود"});
    await q("INSERT INTO repair_status_history(shop_id,repair_id,status,changed_by)
    let whatsapp=null;
    if(status==='تم اصلاحه'){
      const phone=(await one('SELECT phone FROM customers WHERE id=$1',[r.customer_
      const msg=`رقم الوصل:\nأصبح جاهزاً للاستلام BN SMART مرحباً، نعلمكم أن هاتفكم في`
      await q(`INSERT INTO notifications(shop_id,repair_id,channel,message,created_
      const digits=String(phone).replace(/\D/g,'');
      const intl=digits.startsWith('0')?'213'+digits.slice(1):digits;
      whatsapp={phone:intl,message:msg,url:`https://wa.me/${intl}?text=${encodeURIComponent
    }
    await log(req.user.id,"تغيير حالة الإصلاح",r.id,{status});
    res.json({...r,whatsapp});
  });

  app.patch("/api/repairs/:id/received",auth,can("edit_repair"),async(req,res)=>{
    try{
      const raw=req.body?.received;
      const received = raw===true || raw==="true" || raw===1 || raw==="1";
      const r=await one(
        `UPDATE repair_orders
        SET customer_received=$1,
        received_at=CASE WHEN $1 THEN COALESCE(received_at,NOW()) ELSE NULL EN
        updated_by=$2, updated_at=NOW()

```

```

        WHERE id=$3 RETURNING *`,
        [received, req.user.id, req.params.id]
    );
    if(!r) return res.status(404).json({error:"الوصل غير موجود"});
    // Logging must never turn a successful status update into a 500 response.
    try{
        await log(req.user.id, received?"إلغاء تأكيد استلام الجهاز": "لاز من طرف الزبون");
    } catch(logErr){ console.error("pickup log error:", logErr); }
    res.json(r);
} catch(e){
    console.error("pickup update error:", e);
    res.status(500).json({error:"تعذر تحديث حالة الاستلام", detail: process.env.NODE_
});
});

app.post("/api/repairs/:id/payments", auth, can("edit_repair"), async (req, res) => {
    const amount = Number(req.body.amount) || 0;
    if (amount <= 0) return res.status(400).json({error:"قيمة الدفعة غير صحيحة"});
    const client = await pool.connect();
    try{
        await client.query("BEGIN");
        await setTenant(client, req.user.shop_id);
        const r = (await client.query("UPDATE repair_orders SET paid_amount=paid_amount
        if(!r){await client.query("ROLLBACK"); return res.status(404).json({error:"جود
        await client.query("INSERT INTO payments(shop_id, repair_id, amount, recorded_by
        await client.query("COMMIT"); await log(req.user.id, "تسجيل دفعة", r.id, {amount}
        } catch(e){await client.query("ROLLBACK"); res.status(500).json({error:"يل الدفعة
    });

app.get("/api/customers", auth, can("customers"), async (req, res) => {
    res.json(await q(`SELECT c.id, c.name, c.phone, COUNT(r.id)::int repairs_count, COA
    FROM customers c LEFT JOIN repair_orders r ON r.customer_id=c.id GROUP BY c.id
    ));

app.post("/api/customers", auth, can("customers"), async (req, res) => {
    const name = String(req.body?.name || "").trim();
    const phone = String(req.body?.phone || "").trim();
    if (!name || !phone) return res.status(400).json({error:"هاتف مطلوبان"});
    try{
        const existing = await one("SELECT id, name, phone FROM customers WHERE shop_id=$
        if (existing) return res.status(409).json({error:"لهاتف مسجل مسبقاً لهذا الزبون"});
        const c = await one("INSERT INTO customers(shop_id, name, phone) VALUES ($1, $2, $3)
        await log(req.user.id, "إضافة زبون", null, {customer_id: c.id});
        res.status(201).json(c);
    } catch(e){ console.error(e); res.status(500).json({error:"تعذر إضافة الزبون"}) }
    });

```

```

app.post("/api/auth/change-password", auth, async(req,res)=>{
    const {currentPassword,newPassword}=req.body||{};
    if(!currentPassword || !newPassword) return res.status(400).json({error:"لجديدة"
    if(String(newPassword).length < 8) return res.status(400).json({error:"على الأقل
    const u=await one("SELECT * FROM users WHERE id=$1",[req.user.id]);
    if(!u || !u.active || !(await bcrypt.compare(String(currentPassword),u.password
    const hash=await bcrypt.hash(String(newPassword),12);
    await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2",[hash,u.id]);
    await log(req.user.id,"تغيير كلمة المرور",null);
    res.json({ok:true});
});

app.get("/api/users",auth,manager,async(req,res)=>{
    res.json(await q("SELECT id,username,full_name,phone,role,active,permissions,la
});

app.post("/api/users",auth,manager,async(req,res)=>{
    const b=req.body;if(!b.username||!b.fullName||!b.password)return res.status(400
    const hash=await bcrypt.hash(b.password,12);
    const permissions={...DEFAULT_PERMS,...(b.permissions||{})};
    try{
        const u=await one("INSERT INTO users(shop_id,username,full_name,phone,passwor
        [req.user.shop_id,b.username,b.fullName,b.phone|","",hash,JSON.stringify(per
        await log(req.user.id,"إضافة متعاون",null,{username:b.username});res.status(2
    }catch(e){res.status(409).json({error:"المستخدم مستخدم بالفعل"})})
});

app.patch("/api/users/:id",auth,manager,async(req,res)=>{
    const b=req.body;let u=await one("SELECT * FROM users WHERE id=$1",[req.params.
    const username=b.username==null?u.username:String(b.username).trim();
    if(!username)return res.status(400).json({error:"المستخدم مطلوب"});
    const role=b.role==null?u.role:String(b.role);
    if(!["manager","collaborator"].includes(role))return res.status(400).json({erro
    const active=b.active==null?u.active:Boolean(b.active);
    if((u.role!==role || u.active!==active) && (u.role==="manager") && (role!=="man
        const count=Number((await one("SELECT COUNT(*)::int n FROM users WHERE role='
        if(count<=1)return res.status(400).json({error:"على الأقل
    }
    const perms=role==="manager"?JSON.stringify(Object.fromEntries(Object.keys(DEFA
    const passwordHash=b.password?await bcrypt.hash(b.password,12):u.password_hash;
    try{
        u=await one("UPDATE users SET username=$1,full_name=COALESCE($2,full_name),ph
        [username,b.fullName,b.phone,role,active,perms,passwordHash,req.params.id])
        await log(req.user.id,"تعديل حساب مستخدم",null,{user_id:u.id,username:u.usern
    }catch(e){
        if(e && e.code==="23505") return res.status(409).json({error:"مستخدم بالفعل
        console.error(e);res.status(500).json({error:"تعذر تعديل الحساب"});
    }
});

```

```

app.post("/api/users/:id/password", auth, manager, async (req, res) => {
  const b = req.body || {};
  if (!b.newPassword || String(b.newPassword).length < 8) return res.status(400).json({});
  const u = await one("SELECT id, username, role FROM users WHERE id=$1", [req.params.id]);
  if (!u) return res.status(404).json({error: "المستخدم غير موجود"});
  const hash = await bcrypt.hash(String(b.newPassword), 12);
  await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2", [hash, u.id]);
  await log(req.user.id, "تغيير كلمة مرور مستخدم", null, {user_id: u.id, username: u.username});
  res.json({ok: true});
});

app.put("/api/users/:id/password", auth, manager, async (req, res) => {
  const b = req.body || {};
  if (!b.newPassword || String(b.newPassword).length < 8) return res.status(400).json({});
  const u = await one("SELECT id, username, role FROM users WHERE id=$1", [req.params.id]);
  if (!u) return res.status(404).json({error: "المستخدم غير موجود"});
  try {
    const hash = await bcrypt.hash(String(b.newPassword), 12);
    await pool.query("UPDATE users SET password_hash=$1 WHERE id=$2", [hash, u.id]);
    await log(req.user.id, "تغيير كلمة مرور مستخدم", null, {user_id: u.id, username: u.username});
    res.json({ok: true});
  } catch (e) { console.error(e); res.status(500).json({error: "تعذر تغيير كلمة المرور"}); }
});

app.delete("/api/users/:id", auth, manager, async (req, res) => {
  if (Number(req.params.id) === req.user.id) return res.status(400).json({error: "حالي"});
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    await setTenant(client, req.user.shop_id);
    const u = (await client.query("SELECT id, username, full_name, role FROM users WHERE id=$1", [req.params.id]));
    if (!u) return res.status(404).json({error: "المستخدم غير موجود"});
    if (u.role === "manager") {
      const managers = Number((await client.query("SELECT COUNT(*)::int n FROM users WHERE role='manager'")));
      if (managers < 1) return res.status(403).json({error: "أن يوجد مدير آخر فعال أولاً"});
    }
    const label = `${u.full_name} (${u.username})`;
    await client.query("UPDATE activity_logs SET user_id=NULL, details=jsonb_set('action', 'delete', true)");
    await client.query("UPDATE repair_orders SET created_by=NULL, updated_by=NULL");
    await client.query("UPDATE repair_status_history SET changed_by=NULL WHERE changed_by=$1", [u.id]);
    await client.query("UPDATE payments SET recorded_by=NULL WHERE recorded_by=$1", [u.id]);
    await client.query("UPDATE repair_parts SET added_by=NULL WHERE added_by=$1", [u.id]);
    await client.query("UPDATE inventory_transactions SET performed_by=NULL WHERE performed_by=$1", [u.id]);
    await client.query("UPDATE notifications SET created_by=NULL WHERE created_by=$1", [u.id]);
    await client.query("DELETE FROM users WHERE id=$1", [u.id]);
    await client.query("INSERT INTO activity_logs (shop_id, user_id, action, details) VALUES ($1, $2, $3, $4)", [req.user.shop_id, req.user.id, 'delete', label]);
    await client.query("COMMIT");
    res.json({ok: true});
  } catch (e) { console.error(e); res.status(500).json({error: "تعذر حذف المستخدم"}); }
});

```

```

    }catch(e){await client.query("ROLLBACK");console.error(e);res.status(500).json(
    finally{client.release()}}
  });
app.get("/api/activity",auth,manager,async(req,res)=>{
  res.json(await q(`SELECT a.*,u.full_name user_name,r.receipt_no FROM activity_1
  ));

const SUPPLIER_NAMES=["BN SMART","MONTEL","AABIDIN","HASSAN","RAHT ELBAL"];
const SUPPLIER_ITEMS=["LCD","BAT","NAP CHARGE","GLASS","SERSOU","CONCTOUR"];
app.get("/api/suppliers",auth,can("inventory"),async(req,res)=>{
  try{const rows=await q(`SELECT supplier,COUNT(*)::int purchases_count,COALESCE(
  ));
app.get("/api/suppliers/:supplier/purchases",auth,can("inventory"),async(req,res)
app.post("/api/suppliers/purchases",auth,can("inventory"),async(req,res)=>{const
app.delete("/api/suppliers/purchases/:id",auth,manager,async(req,res)=>{try{const

app.get("/api/settings",auth,can("settings"),async(req,res)=>{
  const value=(await one("SELECT value FROM settings WHERE key='shop'"))?.value||
  res.json({...value,name:"BN SMART"});
});
app.put("/api/settings",auth,manager,async(req,res)=>{
  const current=(await one("SELECT value FROM settings WHERE key='shop'"))?.value
  const next={...current,...req.body,name:"BN SMART"};
  await pool.query("INSERT INTO settings(shop_id,key,value) VALUES($1,'shop',$2)
  await log(req.user.id,"تحديث إعدادات المحل");res.json(next);
});

app.get("/api/backup",auth,manager,async(req,res)=>{
  const data={customers:await q("SELECT * FROM customers"),repairs:await q("SELEC
  res.json({exported_at:new Date().toISOString(),data});
});

app.get("/api/track/:code",async(req,res)=>{
  const r=await one(`SELECT * FROM public_track_repair($1)`,[req.params.code]);
  if(!r)return res.status(404).json({error:"رقم التتبع غير موجود"});
  const history=await one(`SELECT public_track_history($1) AS rows`,[r.id]);
  const historyRows=history?.rows||[];
  res.json({receipt_no:r.receipt_no,tracking_code:r.tracking_code,device:[r.brand
  });
app.get("/api/qr/:code",async(req,res)=>{try{const base=`${req.protocol}://${req.

app.get("/track/:code",(req,res)=>res.sendFile(path.join(__dirname,"public","trac
app.get("/share/:code",(req,res)=>{res.set("Cache-Control","no-store, no-cache, m
app.get("*",(req,res)=>res.sendFile(path.join(__dirname,"public","index.html")));

init().then(()=>app.listen(PORT,()=>console.log(`BN SMART running on http://local

```

